МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ • КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ И ПРОГРАММИРОВАНИЯ

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

Тема: «Списочные структуры данных и алгоритм сортировочной станции»

Дисциплина: Алгоритмы и структуры данных
Преподаватель: Молдовян Д.Н.
Выполнил(а): ст. гр. 5372 Павленко П.С.

1. ПОСТАНОВКА ЗАДАЧИ И ЦЕЛЬ РАБОТЫ

Задание к лабораторной работе №1 («Списочные структуры»):

Реализовать базовые линейные структуры данных: **двусвязный список**, **динамический массив** и **стек**. Стек можно реализовать как на базе списка, так и отдельно. Использовать стек для реализации **алгоритма сортировочной станции** (перевод математического выражения в обратную польскую нотацию).
Разрешённые символы в исходном выражении: `+`, `-`, `*`, `/`, `^`, `sin`, `cos`, `(`, `)`, цифры `0..9`. Для упрощения разбиения входной строки на токены разрешается отделять каждый символ пробелом.

2. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ И АРХИТЕКТУРА СТРУКТУР

Линейные динамические структуры обеспечивают эффективное управление памятью во время выполнения программы. В рамках работы спроектированы три фундаментальных контейнера:

- **Двусвязный список (Doubly Linked List):** состоит из узлов (Node), каждый из которых хранит информационное поле `data`, указатель на следующий узел `next` и на предыдущий узел `prev`. Вставка и удаление элементов в начале, конце или по указателю выполняются за константное время **O(1)**, а поиск элемента по индексу — за линейное время **O(n)**.
- **Динамический массив (Dynamic Array):** представляет собой непрерывный блок памяти, автоматически расширяющийся (обычно в 2 раза: `capacity × 2`) при заполнении. Обеспечивает мгновенный произвольный доступ по индексу за **O(1)** и амортизированное добавление в конец за **O(1)**.
- **Стек (Stack — LIFO):** структура, работающая по принципу *«последним пришёл — первым ушёл»*. В данной работе стек построен на базе связного списка, что исключает затраты на реаллокацию памяти и гарантирует выполнение операций `push()`, `pop()` и `peek()` строго за **O(1)**.



Рис. 1. Архитектура узлов двусвязного списка с указателями prev/next и буфера динамического массива

3. ОПИСАНИЕ АЛГОРИТМА СОРТИРОВОЧНОЙ СТАНЦИИ Э. ДЕЙКСТРЫ

Алгоритм сортировочной станции (*Shunting-yard algorithm*) предназначен для преобразования инфиксных математических выражений в постфиксную форму — обратную польскую запись (ОПЗ / RPN), не требующую скобок при вычислении. В процессе разбора используются стек операторов и выходной динамический массив токенов:

1. Если токен — **число** (состоящее из цифр `0..9`), оно сразу помещается в выходной массив.
2. Если токен — **префиксная функция** (`sin`, `cos`) или **открывающая скобка** `(`, он помещается в стек.
3. Если токен — **бинарный оператор** op1: пока на вершине стека находится функция или оператор op2, и при этом (op1 левоассоциативен и его приоритет ≤ приоритету op2) ИЛИ (op1 правоассоциативен и его приоритет < приоритета op2), op2 перекладывается из стека в выходной массив. Затем op1 кладется в стек.
4. Если токен — **закрывающая скобка** `)`: операторы извлекаются из стека в выходной массив до встречи с открывающей скобкой `(`. Сама скобка `(` удаляется, а если после неё на вершине стека лежит функция (`sin`, `cos`), она также переносится в выходной массив.
5. По окончании чтения строки все оставшиеся в стеке операторы поочередно выталкиваются в выходной массив.

Категория	Токены	Приоритет	Ассоциативность	Описание и особенности
Функции	`sin`, `cos`	**4 (высший)**	Префиксные унарные	Применяются к аргументу в скобках
Степень	`^`	**3 (высокий)**	Правая (Right)	Выражение `2 ^ 3 ^ 2` трактуется как $2^{(3^2)}$
Мультипликат.	`*`, `/`	**2 (средний)**	Левая (Left)	Умножение и деление чисел
Аддитивные	`+`, `-`	**1 (низкий)**	Левая (Left)	Сложение и вычитание
Разделители	`(`, `)`	**0 (барьер)**	Не применимо	Управляют порядком свертки подвыражений

4. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СТРУКТУР ДАННЫХ (C++)

```
// 1. Узел и класс двусвязного списка (Doubly Linked List) template <typename T> struct Node { T data; Node* prev; Node* next; Node(const T& v) : data(v), prev(nullptr), next(nullptr) {} }; template <typename T> class DoublyLinkedList { Node<T>* head = nullptr; Node<T>* tail = nullptr; size_t sz = 0; public: ~DoublyLinkedList() { while (!empty()) pop_front(); } bool empty() const { return sz == 0; } void push_front(const T& val) { Node<T>* n = new Node<T>(val); if (!head) head = tail = n; else { n->next = head; head->prev = n; head = n; } sz++; } void push_back(const T& val) { Node<T>* n = new Node<T>(val); if (!tail) head = tail = n; else { tail->next = n; n->prev = tail; tail = n; } sz++; } T pop_front() { Node<T>* temp = head; T val = temp->data; head = head->next; if (head) head->prev = nullptr; else tail = nullptr; delete temp; sz--; return val; } const T& front() const { return head->data; } }; // 2. Класс динамического массива и 3. Стек на базе двусвязного списка template <typename T> class DynamicArray { T* arr; size_t sz = 0, cap = 4; public: DynamicArray() { arr = new T[cap]; } ~DynamicArray() { delete[] arr; } void push_back(const T& val) { if (sz == cap) { cap *= 2; T* nArr = new T[cap]; for (size_t i = 0; i < sz; ++i) nArr[i] = arr[i]; delete[] arr; arr = nArr; } arr[sz++] = val; } size_t size() const { return sz; } T& operator[](size_t i) { return arr[i]; } }; template <typename T> class Stack { DoublyLinkedList<T> list; public: void push(const T& v) { list.push_front(v); } T pop() { return list.pop_front(); } const T& peek() const { return list.front(); } bool empty() const { return list.empty(); } };
```

5. РЕАЛИЗАЦИЯ АЛГОРИТМА СОРТИРОВОЧНОЙ СТАНЦИИ

```
int getPrecedence(const string& op) { if (op == "sin" || op == "cos") return 4; if (op == "^") return 3; if (op == "*" || op == "/") return 2; if (op == "+" || op == "-") return 1; return 0; } bool isRightAssoc(const string& op) { return op == "^"; } bool isFunction(const string& t) { return t == "sin" || t == "cos"; } bool isOperator(const string& t) { return t=="+" || t=="-" || t=="*" || t=="/" || t=="^"; } DynamicArray<string> shuntingYard(const string& expr) { Stack<string> opStack; DynamicArray<string> output; stringstream ss(expr); string token; while (ss >> token) { // Разбиение входной строки на токены, разделенные пробелом if (isdigit(token[0])) { output.push_back(token); } else if (isFunction(token) || token == "(") { opStack.push(token); } else if (isOperator(token)) { while (!opStack.empty() && opStack.peek() != "(" && (isFunction(opStack.peek()) || getPrecedence(opStack.peek()) > getPrecedence(token) || (getPrecedence(opStack.peek()) == getPrecedence(token) && !isRightAssoc(token)))) { output.push_back(opStack.pop()); } opStack.push(token); } else if (token == ")") { while (!opStack.empty() && opStack.peek() != "(") output.push_back(opStack.pop()); if (!opStack.empty()) opStack.pop(); // Удаляем '(' if (!opStack.empty() && isFunction(opStack.peek())) output.push_back(opStack.pop()); } } while (!opStack.empty()) output.push_back(opStack.pop()); return output; }
```

6. ТРАССИРОВКА И РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Ниже приведена пошаговая таблица работы стека и динамического массива для контрольного выражения с функциями, скобками и правоассоциативным возведением в степень: `3 + 4 * 2 / ( 1 - 5 ) ^ 2 + sin ( 0 )`.

Шаг	Входной токен	Действие алгоритма	Стек (дно → верх)	Выходной массив (ОПЗ)
1–3	3, +, 4	Числа в выход, оператор + в стек	+	3 4
4–5	*, 2	Приоритет * > + (в стек), 2 в выход	+ *	3 4 2
6	/	Приоритет * == / (левоассоц.): вытолкнуть *	+ /	3 4 2 *
7–10	(1 - 5	Скобка и - в стек, числа 1 и 5 в выход	+ / (-	3 4 2 * 1 5
11	)	Свертка до (: вытолкнуть -, удалить (	+ /	3 4 2 * 1 5 -
12–13	^, 2	Приоритет ^ (3) > / (2): ^ в стек, 2 в выход	+ / ^	3 4 2 * 1 5 - 2
14	+	Вытолкнуть ^, /, +; положить новый +	+	3 4 2 * 1 5 - 2 ^ / +
15–18	sin (0)	sin и (в стек, 0 в выход; при) вытолкнуть sin	+	3 4 2 * 1 5 - 2 ^ / + 0 sin
19	*Конец*	Выталкивание остатка стека (+) в выходной массив	*пуст*	**3 4 2 * 1 5 - 2 ^ / + 0 sin +**

7. ВЫВОДЫ ПО РАБОТЕ

В ходе выполнения лабораторной работы № 1 были самостоятельно разработаны на языке C++ шаблонные классы **двусвязного списка**, **динамического массива** и **стека**. На базе созданного стека успешно реализован алгоритм сортировочной станции Э. Дейкстры, корректно обрабатывающий арифметические операторы с различными приоритетами, правоассоциативное возведение в степень (^), скобочные группы и тригонометрические функции (sin, cos). Вычислительная сложность разбора